



Elliptic Curves Cryptography (ECC)

ECDSA

Mihai TOGAN
mihai.togan@mta.ro

Agenda



- **EC Cryptography (ECC)**
- **ECDSA**



- Most of public-key crypto systems (RSA, D-H) use either integer or polynomial arithmetic with very large numbers/polynomials
 - Imposes a significant load in storing and processing keys and messages

- An alternative is to use the arithmetic of the elliptic curves
 - Basically, it is “*an approach to public-key cryptography based on the mathematics of elliptic curves*” (Wikipedia)

 - Pro: offers same security with smaller bit sizes
 - Contra: they are newer and not as well analyzed



Elliptic Curves – Definitions

- An elliptic curve over real numbers may be defined as the set of points (x, y) which satisfy an elliptic curve equation of the form: $y^2 = x^3 + ax + b$
 - x, y, a, b are real numbers
 - a, b are the coefficients
 - x, y are the variables
 - define also a special point O called the point at infinity (zero point)

- **Note:** each choice of the numbers a & b yields a different elliptic curve



Elliptic Curves

Definition 3.1 An *elliptic curve* E over a field K is defined by an equation

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad (3.1)$$

where $a_1, a_2, a_3, a_4, a_6 \in K$ and $\Delta \neq 0$, where Δ is the *discriminant* of E and is defined as follows:

$$\left. \begin{aligned} \Delta &= -d_2^2d_8 - 8d_4^3 - 27d_6^2 + 9d_2d_4d_6 \\ d_2 &= a_1^2 + 4a_2 \\ d_4 &= 2a_4 + a_1a_3 \\ d_6 &= a_3^2 + 4a_6 \\ d_8 &= a_1^2a_6 + 4a_2a_6 - a_1a_3a_4 + a_2a_3^2 - a_4^2 \end{aligned} \right\} \quad (3.2)$$

If L is any extension field of K , then the set of L -rational points on E is

$$E(L) = \{(x, y) \in L \times L : y^2 + a_1xy + a_3y - x^3 - a_2x^2 - a_4x - a_6 = 0\} \cup \{\infty\}$$

where ∞ is the *point at infinity*.



Remark 3.2 (*comments on Definition 3.1*)

- (i) Equation (3.1) is called a *Weierstrass equation*.
- (ii) We say that E is *defined over* K because the coefficients a_1, a_2, a_3, a_4, a_6 of its defining equation are elements of K . We sometimes write E/K to emphasize that E is defined over K , and K is called the *underlying field*. Note that if E is defined over K , then E is also defined over any extension field of K .
- (iii) The condition $\Delta \neq 0$ ensures that the elliptic curve is “smooth”, that is, there are no points at which the curve has two or more distinct tangent lines.

Elliptic Curves – Examples

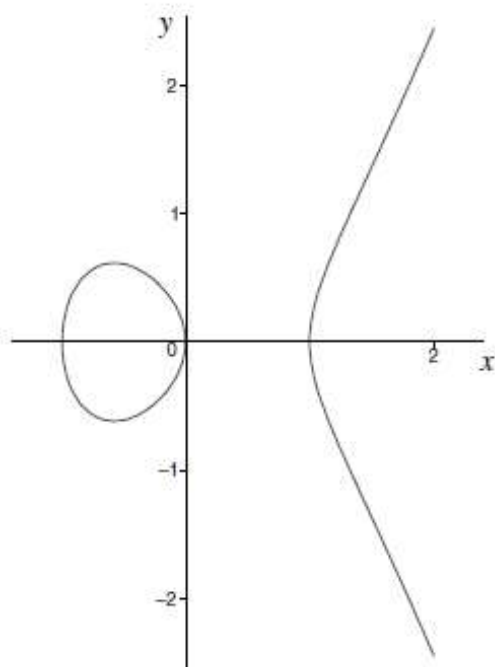


Example (*elliptic curves over $\mathbb{R}$*) Consider the elliptic curves

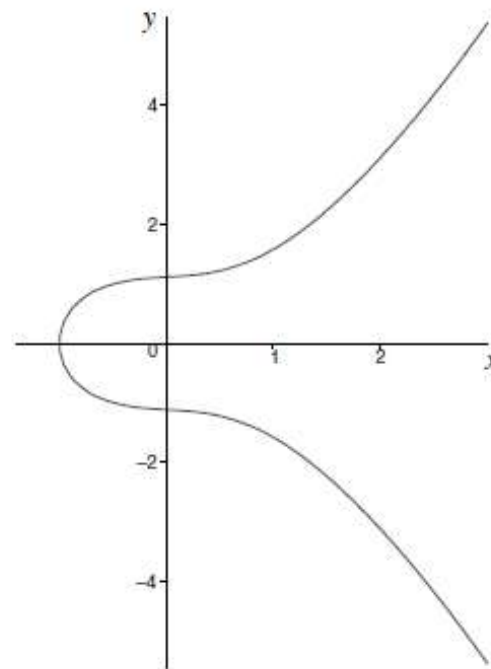
$$E_1 : y^2 = x^3 - x$$

$$E_2 : y^2 = x^3 + \frac{1}{4}x + \frac{5}{4}$$

defined over the field $\mathbb{R}$ of real numbers. The points $E_1(\mathbb{R}) \setminus \{\infty\}$ and $E_2(\mathbb{R}) \setminus \{\infty\}$ are graphed



(a) $E_1 : y^2 = x^3 - x$

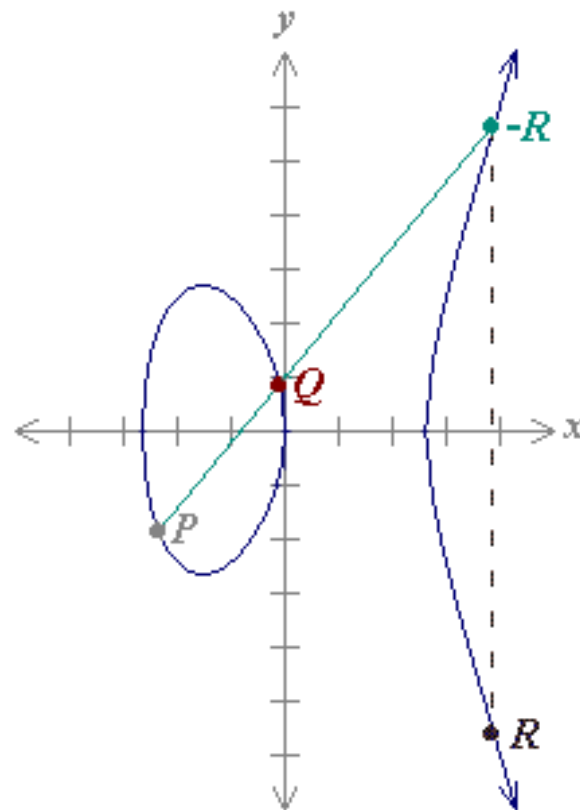


(b) $E_2 : y^2 = x^3 + \frac{1}{4}x + \frac{5}{4}$



EC – Addition operation

- Define addition operation $P + Q$
 - Geometrical, the sum $P + Q$ is the reflection of the intersection R



$$P (-2.35, -1.86)$$

$$Q (-0.1, 0.836)$$

$$-R (3.89, 5.62)$$

$$R (3.89, -5.62)$$

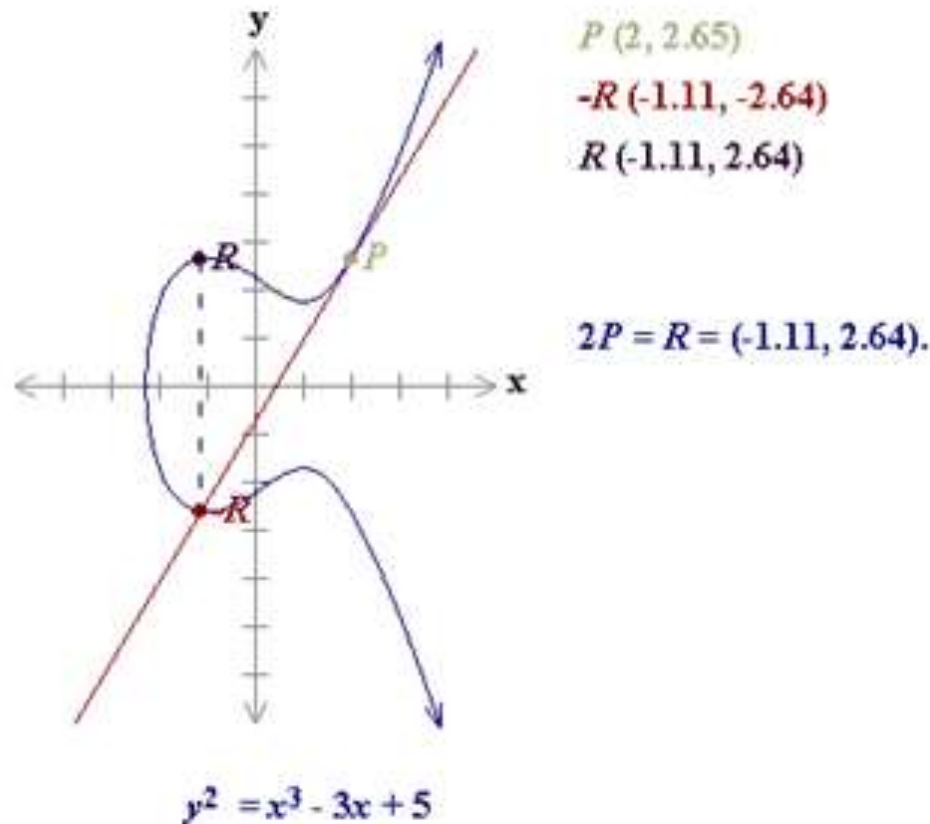
$$P + Q = R = (3.89, -5.62).$$

$$y^2 = x^3 - 7x$$

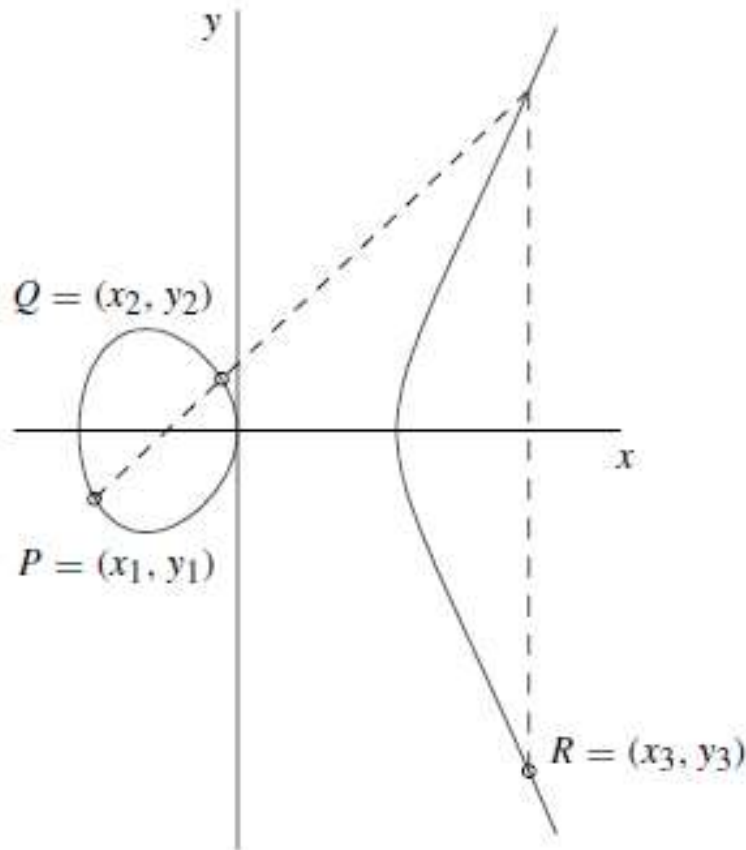


EC – Doubling operation

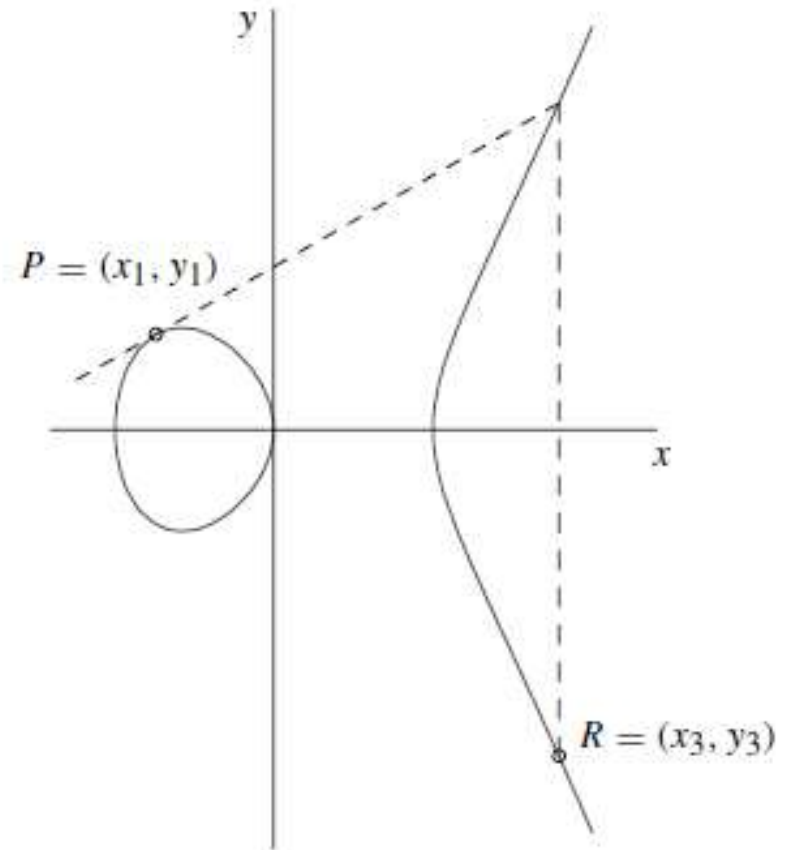
- To add P to itself, a tangent line to the curve is drawn at the point P .
- The tangent line will intersect the elliptic curve at the point $-R$, if the y value of P is not 0. $-R$ is then reflected into the x -axis to provide R



Addition and doubling operations – Examples



(a) Addition: $P + Q = R$.

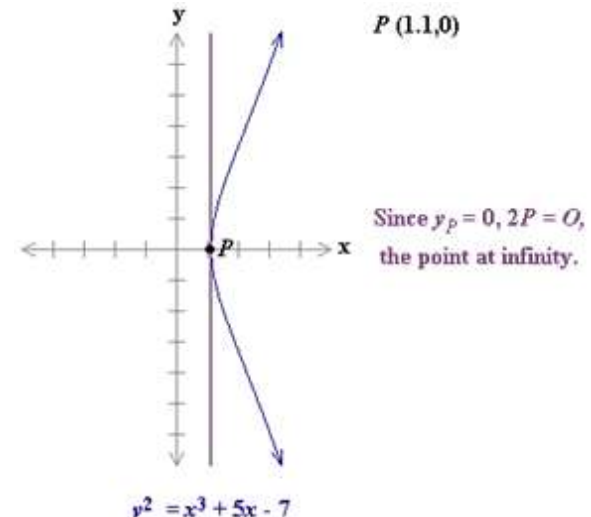
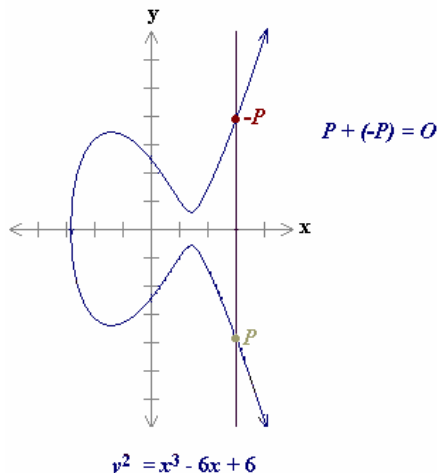


(b) Doubling: $P + P = R$.

EC Operations – Exceptions



- There are two exceptions (where drawing a line through points P and Q will provide point $-R$)
 - The first exception occurs when adding points P and $-P$.
 - Since drawing a line through point P and $-P$ will result in a vertical line (which will not cross through the elliptic curve at a third point), the point at infinity O is needed.
 - By definition, $P + (-P) = O$, therefore, $P + O = P$
 - The second exception occurs when doubling point P
 - If a point P is such that $y_P = 0$, then the tangent line to the elliptic curve at P is vertical and does not intersect the elliptic curve at any other point.
 - By definition, $2P = O$, for such a point P ($y_P = 0$)





EC – Algebraically computations

- **Let** $P = (x_1, y_1)$ **and** $Q = (x_2, y_2)$

- $P + Q = R = (x_3, y_3)$, **where**

- $x_3 = m^2 - x_1 - x_2$

- $y_3 = m \cdot (x_1 - x_3) - y_1$

where

- **if** $P \neq Q$, **then** $m = (y_2 - y_1) / (x_2 - x_1)$

- **if** $P = Q$, **then** $m = (3 \cdot x_1^2 + a) / (2 \cdot y_1)$

Finite Elliptic Curves



- Elliptic curve cryptography uses curves whose coefficients & variables are finite

- There are two families commonly used
 - Prime curves $E_p(a, b)$ defined over $\mathbb{Z}_p$
 - Use integers modulo a prime p
 - Best in software

 - Binary curves $E_{2^m}(a, b)$ defined over $\text{GF}(2^m)$
 - Use polynomials with binary coefficients
 - Best in hardware

EC-based Cryptography (ECC)



- Use finite fields (these have a finite number of points)
- If the finite field of integers modulo a prime number p is used (F_p), therefore:
$$x_3 = m^2 - x_1 - x_2 \pmod{p}$$
$$y_3 = m(x_1 - x_3) - y_1 \pmod{p}$$

where

if $P \neq Q$, then $m = [(y_2 - y_1) / (x_2 - x_1)] \pmod{p}$

if $P = Q$, then $m = [(3x_1^2 + a) / (2y_1)] \pmod{p}$



Prime fields

- Let p be a prime number.
- The integers modulo p , consisting of the integers $\{0, 1, 2, \dots, p-1\}$ with addition and multiplication performed modulo p , **is a finite field of order p**
- We shall denote this field by F_p and call p the *modulus* of F_p .
- For any integer a , $a \bmod p$ shall denote the unique integer remainder r , $0 \leq r \leq p-1$, obtained upon dividing a by p ; this operation is called *reduction modulo p* .
- **Example** (*prime field F_{29}*) The elements of F_{29} are $\{0, 1, 2, \dots, 28\}$.
- The following are some examples of arithmetic operations in F_{29} :
 - (i) Addition: $17+20 = 8$ since $37 \bmod 29 = 8$
 - (ii) Subtraction: $17-20 = 26$ since $-3 \bmod 29 = 26$
 - (iii) Multiplication: $17 \cdot 20 = 21$ since $340 \bmod 29 = 21$
 - (iv) Inversion: $17^{-1} = 12$ since $17 \cdot 12 \bmod 29 = 1$



Binary fields

- Finite fields of order 2^m are called *binary fields* or *characteristic-two finite fields*.
- One way to construct F_2^m is to use a *polynomial basis representation*.
- Here, the elements of F_2^m are the binary polynomials (polynomials whose coefficients are in the field $F_2 = \{0, 1\}$) of degree at most $m - 1$:

$$\mathbb{F}_{2^m} = \{a_{m-1}z^{m-1} + a_{m-2}z^{m-2} + \cdots + a_2z^2 + a_1z + a_0 : a_i \in \{0, 1\}\}$$

- An irreducible binary polynomial $f(z)$ of degree m is chosen (such a polynomial exists for any m and can be efficiently found)
 - Addition of field elements is the usual addition of polynomials, with coefficient arithmetic performed modulo 2.
 - Multiplication of field elements is performed modulo the *reduction polynomial* $f(z)$
- For any binary polynomial $a(z)$, $a(z) \bmod f(z)$ shall denote the unique remainder polynomial $r(z)$ of degree less than m obtained upon long division of $a(z)$ by $f(z)$; this operation is called *reduction modulo $f(z)$*



Binary fields (cont.)

Example (binary field $\mathbb{F}_{2^4}$) The elements of $\mathbb{F}_{2^4}$ are the 16 binary polynomials of degree at most 3:

0	z^2	z^3	$z^3 + z^2$
1	$z^2 + 1$	$z^3 + 1$	$z^3 + z^2 + 1$
z	$z^2 + z$	$z^3 + z$	$z^3 + z^2 + z$
$z + 1$	$z^2 + z + 1$	$z^3 + z + 1$	$z^3 + z^2 + z + 1$

The following are some examples of arithmetic operations in $\mathbb{F}_{2^4}$ with reduction polynomial $f(z) = z^4 + z + 1$.

- (i) Addition: $(z^3 + z^2 + 1) + (z^2 + z + 1) = z^3 + z$.
- (ii) Subtraction: $(z^3 + z^2 + 1) - (z^2 + z + 1) = z^3 + z$. (Note that since $-1 = 1$ in $\mathbb{F}_2$, we have $-a = a$ for all $a \in \mathbb{F}_{2^m}$.)
- (iii) Multiplication: $(z^3 + z^2 + 1) \cdot (z^2 + z + 1) = z^2 + 1$ since

$$(z^3 + z^2 + 1) \cdot (z^2 + z + 1) = z^5 + z + 1$$

and

$$(z^5 + z + 1) \bmod (z^4 + z + 1) = z^2 + 1.$$

- (iv) Inversion: $(z^3 + z^2 + 1)^{-1} = z^2$ since $(z^3 + z^2 + 1) \cdot z^2 \bmod (z^4 + z + 1) = 1$.

The *Hard Problem* for ECC



- Every cryptosystem is based on a hard mathematical problem that is “computationally infeasible to solve”
- ECC relies on the difficulty of solving the Discrete Logarithm Problem for the group of an Elliptic Curve over some finite field (e.g. integers modulo a prime number, or a Galois field of size a power of 2) – ECDLP
- Elliptic curves have cryptographic value because
 - A user can ‘multiply’ a point P by a number k to produce another point $Q = k \cdot P$ on given curve
 - The user cannot easily figure out what number k was used – even with knowledge of the original point P and the result Q

The *Hard Problem* for ECC (cont.)



- Given points P and Q in a group, the *Elliptic Curve Discrete Logarithm Problem (ECDLP)* is to find a number such that $k \cdot P = Q$, for some integer k .
- Known methods for calculating the elliptic curve discrete logarithms are less efficient than those for factoring or calculating conventional logarithms
 - The consequence: shorter key sizes can be used to provide the same level of security as other public-key cryptosystems
 - We can say is that ECC provides the most security per bit of any public key scheme

Comparable Key Sizes for Equivalent Security



Symmetric scheme (key size in bits)	ECC-based scheme (size of n in bits)	RSA/DSA (modulus size in bits)
56	112	512
80	160	1024
112	224	2048
128	256	3072
192	384	7680
256	512	15360



- First, the plaintext message needs to be mapped to a numerical value upon which mathematical operations can be performed;
 - This means that for this use, the message must be mapped to a point on an elliptic curve

- To get the ciphertext, it is necessary to perform elliptic curve arithmetics
 - ECC makes use of the addition operation of elliptic curves (RSA would use modular multiplication)
 - ECC's multiple addition is the equivalent of RSA's modular exponentiation



ECC Domain Parameters

To use ECC all parties must agree on all the elements defining the elliptic curve (the *domain parameters* of the scheme):

- The field is defined by p in the prime case or the pair of m, f in the binary u-case
 - The elliptic curve is defined by the constants a, b used in its defining equation
 - The cyclic subgroup is defined by its generator G (a.k.a. base point).
 - The order of G , that is the smallest positive number n such that $n \cdot G = O$
 - The cofactor $h = 1/n \mid E(Fp)$
-
- Domain parameters for a prime field curve are (p, a, b, G, n, h)
 - Domain parameters for a binary field curve are (m, f, a, b, G, n, h)

ECC Domain Parameters



- Such domain parameters are commonly defined by trust organisations
 - NIST, Recommended Elliptic Curves for Government Use
 - SECG, SEC 2: Recommended Elliptic Curve Domain Parameters
 - ECC Brainpool (RFC 5639), ECC Brainpool Standard Curves and Curve Generation
 -
- In this case, the domain parameters are known as "standard curves" or "named curves"

ECC Domain Parameters



- Example: openssl implementation of the ECs list

```
C:\work\bin>openssl ecparam -list_curves
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
secp112r1 : SECG/WTLS curve over a 112 bit prime field
secp112r2 : SECG curve over a 112 bit prime field
secp128r1 : SECG curve over a 128 bit prime field
secp128r2 : SECG curve over a 128 bit prime field
secp160k1 : SECG curve over a 160 bit prime field
secp160r1 : SECG curve over a 160 bit prime field
secp160r2 : SECG/WTLS curve over a 160 bit prime field
secp192k1 : SECG curve over a 192 bit prime field
secp224k1 : SECG curve over a 224 bit prime field
secp224r1 : NIST/SECG curve over a 224 bit prime field
secp256k1 : SECG curve over a 256 bit prime field
secp384r1 : NIST/SECG curve over a 384 bit prime field
secp521r1 : NIST/SECG curve over a 521 bit prime field
prime192v1 : NIST/X9.62/SECG curve over a 192 bit prime field
prime192v2 : X9.62 curve over a 192 bit prime field
prime192v3 : X9.62 curve over a 192 bit prime field
prime239v1 : X9.62 curve over a 239 bit prime field
prime239v2 : X9.62 curve over a 239 bit prime field
prime239v3 : X9.62 curve over a 239 bit prime field
prime256v1 : X9.62/SECG curve over a 256 bit prime field
sect113r1 : SECG curve over a 113 bit binary field
sect113r2 : SECG curve over a 113 bit binary field
sect131r1 : SECG/WTLS curve over a 131 bit binary field
sect131r2 : SECG curve over a 131 bit binary field
sect163k1 : NIST/SECG/WTLS curve over a 163 bit binary field
sect163r1 : SECG curve over a 163 bit binary field
sect163r2 : NIST/SECG curve over a 163 bit binary field
sect193r1 : SECG curve over a 193 bit binary field
sect193r2 : SECG curve over a 193 bit binary field
sect233k1 : NIST/SECG/WTLS curve over a 233 bit binary field
sect233r1 : NIST/SECG/WTLS curve over a 233 bit binary field
sect239k1 : SECG curve over a 239 bit binary field
sect283k1 : NIST/SECG curve over a 283 bit binary field
sect283r1 : NIST/SECG curve over a 283 bit binary field
sect409k1 : NIST/SECG curve over a 409 bit binary field
sect409r1 : NIST/SECG curve over a 409 bit binary field
sect571k1 : NIST/SECG curve over a 571 bit binary field
sect571r1 : NIST/SECG curve over a 571 bit binary field
c2pnb163v1 : X9.62 curve over a 163 bit binary field
c2pnb163v2 : X9.62 curve over a 163 bit binary field
c2pnb163v3 : X9.62 curve over a 163 bit binary field
c2pnb176v1 : X9.62 curve over a 176 bit binary field
c2pnb181v1 : X9.62 curve over a 181 bit binary field
```

ECC Key Generation



1. Input: the domain parameters of the curve
 - (p, a, b, G, n, h) for a prime field
 - (m, f, a, b, G, n, h) for a binary field
2. Randomly select the private-key, an integer $k \in [1, n - 1]$
3. Compute the public-key (the public point, on the curve) : $Q = k \cdot G$
4. Output the key-pair (k, Q)

ECC Algorithms



- **ECDH – the ECC variant of Diffie-Hellman algorithm**
- **ECDSA – the ECC variant of DSA algorithm**

Digital Signatures using ECDSA



ECDSA signatures generation algorithm:

- The domain parameters are given (we can use a named curve)
- Alice creates a key-pair, consisting of
 - The private-key, the integer d_A randomly selected in the range $[1, n - 1]$;
 - The public-key, the curve point $Q_A = d_A \cdot G$

Alice signs the message m using ECDSA scheme, then she follows the next steps:

1. Calculate $e = h(m)$, where h is a cryptographic function (SHA-2, SHA-3,...)
2. Let z be the L_n leftmost bits of e , where L_n is the bit length of n (the group order).
3. Select a cryptographically secure random integer $k \in [1, n - 1]$.
4. Compute the curve point $R = (x_1, y_1) = k \cdot G$
5. Compute $r = x_1 \pmod n$. If $r = 0$ then go back to step 3
6. Compute $s = k^{-1} \cdot (z + d_A \cdot r) \pmod n$. If $s = 0$, go back to step 3
7. The signature on message m is the pair (r, s)

Digital Signatures using DSA – (to compare with ECDSA)



DSA signatures generation algorithm:

To sign a message $m \in M$, the **USER A** will make the next computations:

1. Compute $h(m)$ using an appropriate collision resistant hash function h
2. Select a random integer number k such that $1 \leq k \leq q - 1$
3. Compute $r = (g^k \pmod p) \pmod q$. If $r = 0$ jump again to step 2
4. Compute $s = (k^{-1} \cdot (h(m) + x \cdot r)) \pmod q$. If $s = 0$ jump again to step 2
5. The signature on message m is the pair (r, s)

Digital Signatures using ECDSA (cont.)



ECDSA signatures verification algorithm:

Bob wants to verify the Alice's signature (r, s) generated on the message m . Then he follows the next steps:

1. Verify that r and s are integers in range $[1, n - 1]$. If not, the signature is **invalid**
2. Calculate $e = h(m)$, where h is the same function used in the signature generation process
3. Let z be the L_n leftmost bits of e
4. Compute $w = s^{-1} \pmod{n}$
5. Compute $u_1 = z \cdot w \pmod{n}$ and $u_2 = r \cdot w \pmod{n}$
6. Compute the curve point $(x_1, y_1) = u_1 \cdot G + u_2 \cdot Q_A$
 - If $(x_1, y_1) = O$ then the signature is **invalid**
7. Compare $r \equiv x_1 \pmod{n}$
 - a) If $r \equiv x_1 \pmod{n}$, then signature (r, s) on the message m is **valid**
 - b) Otherwise the signature is **invalid**



DSA signatures verification algorithm:

To verify a signature (r, s) on the message $m \in M$, any **USER B** will make the next computations:

1. Achieve the signer public-key y and the corresponding domain parameters (p, q, g) . The public-key must be authentic.
2. Check if the r, s values are within the range $[1, q - 1]$
3. Compute $h(m)$ using same hash function h used by the signer
4. Compute $w = s^{-1} \pmod{q}$
5. Compute $u_1 = (h(m) \cdot w) \pmod{q}$ and $u_2 = (r \cdot w) \pmod{q}$
6. Compute $v = ((g^{u_1} \cdot y^{u_2}) \pmod{p}) \pmod{q}$
7. Compare $v = r$
 - a) If $v = r$, then the signature (r, s) on the message m is **valid**.
 - b) Otherwise the signature is **invalid**.

Digital Signatures using ECDSA (cont.)



ECDSA signatures verification algorithm:

The proof:

$$x_1 = \dots = r$$

Notes:

1. Signing operation uses the private key d_A (scalar integer)
2. Verification operation uses public key QA (point)
3. You can NOT find d_A knowing $(QA, (EC \text{ domain parameters}), (r, s))$ (ECDLP)
4. You can NOT find k knowing $(QA, (EC \text{ domain parameters}), (r, s))$ (ECDLP)

Generate an ECC key (using openssl)



C:\Windows\system32\cmd.exe

```
C:\work\bin>openssl ecparam -name prime256v1 -genkey -out keypair.ecc
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
Loading 'screen' into random state - done

C:\work\bin>openssl ec -in keypair.ecc -text -param_out
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
read EC key
Private-Key: (256 bit)
priv:
    3f:72:df:1d:c0:ea:8d:fd:0e:06:de:89:eb:6f:66:
    7a:6e:90:aa:c4:c3:85:fc:25:17:1c:d7:20:4c:10:
    21:bf
pub:
    04:2b:dd:a9:f4:3f:2e:a2:59:d2:f2:f1:20:6e:25:
    b2:dc:d2:a5:7b:4e:f2:ec:61:fe:ea:4f:7c:c7:11:
    03:75:45:ef:ac:3c:3f:82:4f:06:0c:fe:a8:34:16:
    fe:6c:b0:f8:a1:04:1f:b5:f7:18:76:17:15:aa:c1:
    d0:bb:70:f8:fd
ASN1 OID: prime256v1
writing EC key
-----BEGIN EC PARAMETERS-----
BggqhkjOPQMBBw==
-----END EC PARAMETERS-----

C:\work\bin>_
```

EC Key & Params



C:\Windows\system32\cmd.exe

```
C:\work\bin>openssl ec -in keypair.ecc -text -param_enc explicit
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
read EC key
Private-Key: (256 bit)
priv:
    3f:72:df:1d:c0:ea:8d:fd:0e:06:de:89:eb:6f:66:
    7a:6e:90:aa:c4:c3:85:fc:25:17:1c:d7:20:4c:10:
    21:bf
pub:
    04:2b:dd:a9:f4:3f:2e:a2:59:d2:f2:f1:20:6e:25:
    b2:dc:d2:a5:7b:4e:f2:ec:61:fe:ea:4f:7c:c7:11:
    03:75:45:ef:ac:3c:3f:82:4f:06:0c:fe:a8:34:16:
    fe:6c:b0:f8:a1:04:1f:b5:f7:18:76:17:15:aa:c1:
    d0:bb:70:f8:fd
Field Type: prime-field
Prime:
    00:ff:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:
    00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:ff:
    ff:ff:ff
A:
    00:ff:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:
    00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:ff:
    ff:ff:fc
B:
    5a:c6:35:d8:aa:3a:93:e7:b3:eb:bd:55:76:98:86:
    bc:65:1d:06:b0:cc:53:b0:f6:3b:ce:3c:3e:27:d2:
    60:4b
Generator (uncompressed):
    04:6b:17:d1:f2:e1:2c:42:47:f8:bc:e6:e5:63:a4:
    40:f2:77:03:7d:81:2d:eb:33:a0:f4:a1:39:45:d8:
    98:c2:96:4f:e3:42:e2:fe:1a:7f:9b:8e:e7:eb:4a:
    7c:0f:9e:16:2b:ce:33:57:6b:31:5e:ce:cb:b6:40:
    68:37:bf:51:f5
Order:
    00:ff:ff:ff:ff:ff:00:00:00:00:ff:ff:ff:ff:ff:ff:
    ff:ff:bc:e6:fa:ad:a7:17:9e:84:f3:b9:ca:c2:fc:
    63:25:51
Cofactor: 1 (0x1)
Seed:
    c4:9d:36:08:86:e7:04:93:6a:66:78:e1:13:9d:26:
    b7:81:9f:7e:90
writing EC key
-----BEGIN EC PRIVATE KEY-----
MIIBAgIBAQgP3LfHcDqjf00Bt6J629mem6QqsTDhfWlFxzXIEwQIb+ggfowgfcC
AQEwLAYHKOZlZj0BAQIhAP////8AAAAABAAAAAAAAAAAAAAAAAAAAA////////
MFsEIP////8AAAAABAAAAAAAAAAAAAAAAAAAAA////////8BCBaxjXyqjqT57Pr
uUU2mIa8ZB0GSMxTsPVZzjw+I9JgSwMUAMSDNqIG5wSTamZ44R0dIreBn36QREFF
```

```
C:\work\bin>openssl ec -in keypair.ecc -pubout -out keypub.ecc
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
read EC key
writing EC key

C:\work\bin>openssl ec -pubin -in keypub.ecc -text -param_enc explicit
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
read EC key
Private-Key: (256 bit)
pub:
    04:2b:dd:a9:f4:3f:2e:a2:59:d2:f2:f1:20:6e:25:
    b2:dc:d2:a5:7b:4e:f2:ec:61:fe:ea:4f:7c:c7:11:
    03:75:45:ef:ac:3c:3f:82:4f:06:0c:fe:a8:34:16:
    fe:6c:b0:f8:a1:04:1f:b5:f7:18:76:17:15:aa:c1:
    d0:bb:70:f8:fd
Field Type: prime-field
Prime:
    00:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:00:
    00:00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:
    ff:ff:ff
A:
    00:ff:ff:ff:ff:00:00:00:01:00:00:00:00:00:00:
    00:00:00:00:00:00:ff:ff:ff:ff:ff:ff:ff:ff:ff:
    ff:ff:fc
B:
    5a:c6:35:d8:aa:3a:93:e7:b3:eb:bd:55:76:98:86:
    bc:65:1d:06:b0:cc:53:b0:f6:3b:ce:3c:3e:27:d2:
    60:4b
Generator (uncompressed):
    04:6b:17:d1:f2:e1:2c:42:47:f8:bc:e6:e5:63:a4:
    40:f2:77:03:7d:81:2d:eb:33:a0:f4:a1:39:45:d8:
    98:c2:96:4f:e3:42:e2:fe:1a:7f:9b:8e:e7:eb:4a:
    7c:0f:9e:16:2b:ce:33:57:6b:31:5e:ce:cb:b6:40:
    68:37:bf:51:f5
Order:
    00:ff:ff:ff:ff:00:00:00:00:ff:ff:ff:ff:ff:ff:
    ff:ff:bc:e6:fa:ad:a7:17:9e:84:f3:b9:ca:c2:fc:
    63:25:51
Cofactor: 1 (0x1)
Seed:
    c4:9d:36:08:86:e7:04:93:6a:66:78:e1:13:9d:26:
    b7:81:9f:7e:90
writing EC key
-----BEGIN PUBLIC KEY-----
MIIBSzCCAQMGBYqGSM49AgEwgfcCAQEwLAYHKOZIzj0BAQIhAP///8AAAAABAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
////////////////////MFsEIP///8AAAAABAAAAAAAAAAAAAAAAAAAAAAAAAAAA
////////////////8BCBaxjXyqJgT57PrvUU2mIa8ZR0GsMxTsPY7zjw+J9JgSwMUAMSd
NriG5uSTamZ44R0d.IveBo360BEFEayfR8uEsQkf4uOb1V6R08ncDfVEt6z0c9KE5
```

Sign & Verify



C:\Windows\system32\cmd.exe

```
C:\work\bin>openssl pkeyutl -sign -inkey keypair.ecc -in file1.txt -out file1.ecdsa.sig
```

```
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
```

```
Loading 'screen' into random state - done
```

```
C:\work\bin>openssl pkeyutl -verify -pubin -inkey keypub.ecc -in file1.txt -sigfile file1.ecdsa.sig
```

```
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
```

```
Loading 'screen' into random state - done
```

```
Signature Verified Successfully
```

```
C:\work\bin>_
```

Sign & Verify – two sign sessions (!!)



C:\Windows\system32\cmd.exe

```
C:\work\bin>openssl pkeyutl -sign -inkey keypair.ecc -in file1.txt -out file1.ecdsa.sig
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
Loading 'screen' into random state - done
```

```
C:\work\bin>openssl pkeyutl -verify -pubin -inkey keypub.ecc -in file1.txt -sigfile file1.ecdsa.sig
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
Loading 'screen' into random state - done
Signature Verified Successfully
```

```
C:\work\bin>openssl asn1parse -inform DER -in file1.ecdsa.sig -i
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
 0:d=0  hl=2 l= 69 cons: SEQUENCE
 2:d=1  hl=2 l= 32 prim: INTEGER           :21B4179AA7394651A4625B931B7DA8941DD098B0DA655ED326AB7D01CCE5C491
36:d=1  hl=2 l= 33 prim: INTEGER           :E1A20005400098609F2A148263D75882E8D58D017A95540F6C82F44D9A4DCCC0
```

```
C:\work\bin>openssl pkeyutl -sign -inkey keypair.ecc -in file1.txt -out file1.ecdsa.sig
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
Loading 'screen' into random state - done
```

```
C:\work\bin>openssl pkeyutl -verify -pubin -inkey keypub.ecc -in file1.txt -sigfile file1.ecdsa.sig
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
Loading 'screen' into random state - done
Signature Verified Successfully
```

```
C:\work\bin>openssl asn1parse -inform DER -in file1.ecdsa.sig -i
WARNING: can't open config file: e:\utile\openssl\ssl\openssl.cnf
 0:d=0  hl=2 l= 69 cons: SEQUENCE
 2:d=1  hl=2 l= 33 prim: INTEGER           :82CED9C1D957FCDF100BC8E2B9FFF041D26607FA3D0040788899098ADAF9A8BC
37:d=1  hl=2 l= 32 prim: INTEGER           :01FA865A87190A6F499897A39F042294529429C6A7331596EB13A09C82B1A99A
```

```
C:\work\bin>_
```



Bibliography:

D. Hankerson, A. Menezes, S. Vanstone,
Guide to Elliptic Curve Cryptography



ECDH (key agreement)

- The domain parameters are given (we can use a named curve)
- Alice and Bob select their private keys, $n_A < n$ and $n_B < n$
- Alice and Bob compute their corresponding public keys
 - Alice: $P_A = n_A \cdot G$
 - Bob: $P_B = n_B \cdot G$
- Alice and Bob switch their public-keys
- Each compute the same shared key $K = n_A \cdot P_B$, $K = n_B \cdot P_A$
 - since $K = n_A \cdot n_B \cdot G = n_B \cdot n_A \cdot G$
- Note: the attacker would need to find K, hard (!!)